

서비스 & 하드웨어 전통적 SfM + MVS

Junha Lee

Architecture and Compiler for Embedded System LAB.

School of Electronics Engineering, KNU, KOREA

2026-06-30



목차

- 커스텀 메시지와 서비스의 필요성
- Service 통신 구현
- 전통적 SfM + MVS

커스텀 메시지와 서비스의 필요성

- 표준 메시지(std_msgs 등)만으로는 의미 있는 데이터를 묶어 표현하기 어려움
- 커스텀 인터페이스 : 도메인에 특화된 데이터 구조를 직접 정의
- 메시지 (.msg)
 - 단방향 · 비동기 — 응답을 기대하지 않음 (Topic)
- 서비스 (.srv)
 - 양방향 · 동기 — 결과를 받을 때까지 대기 (Service)
- IDL로 한 번 정의하면 Python · C++ 등 언어별 코드가 자동 생성
 - 서로 다른 언어로 작성한 노드끼리도 같은 메시지를 주고받음

ROS2 인터페이스 종류

- 메시지 (.msg)
 - 토픽(Topic)에서 사용하는 단방향 데이터 전달
- 서비스 (.srv)
 - 요청(Request)과 응답(Response)으로 구성된 동기 통신
- 액션 (.action)
 - 목표 · 피드백 · 결과로 구성, 시간이 걸리는 작업에 사용
- 빌드 시 rosidl이 각 인터페이스를 언어별 소스코드로 변환

Topic vs Service

- 통신 방식
 - Topic : 단방향 ($1 \rightarrow N$)
 - Service : 양방향 ($1 : 1$)
- 동기성
 - Topic : 비동기 (연속적 게시)
 - Service : 동기 (요청 후 응답 대기)
- 인터페이스
 - Topic : .msg
 - Service : .srv (--- 로 요청·응답 구분)
- 사용 예
 - Topic : 센서 데이터 스트림
 - Service : 값 계산 · 상태 변경

커스텀 .srv 파일 정의

- .srv 구조 : 요청과 응답을 --- 로 구분

```
int64 a  
int64 b  
---  
int64 sum
```

- --- 위는 요청(Request), 아래는 응답(Response) 필드
- srv/ 폴더에 저장 (예: srv/AddTwoInts.srv)
- 빌드 시 Request · Response 타입이 한 쌍으로 자동 생성

패키지 생성 & 빌드

- 패키지 생성

```
ros2 pkg create --build-type ament_cmake \  
  cpp_srvcli --dependencies rclcpp example_interfaces
```

- dependencies 옵션이 package.xml · CMakeLists.txt에 의존성 자동 추가

- 빌드 & 환경 설정

```
colcon build --packages-select cpp_srvcli  
source install/setup.bash
```

- rosidl 빌드로 .srv 타입이 코드로 변환되어 노드에서 사용

Service 통신 동작 원리

- 클라이언트가 요청을 보내면 서버가 처리 후 응답을 반환 (동기 1:1)



- `create_client` / `create_service` 로 같은 이름 · 타입에 연결
- `wait_for_service` 로 서버 준비를 확인한 뒤 요청 전송
- 서버 콜백이 결과를 채워 응답, 클라이언트가 수신

Service 구현 방법

- 서버 (Server)
 - create_service(타입, 이름, 콜백) 으로 서비스 등록
 - 콜백(request, response) 에서 처리 후 응답 반환
 - spin() 으로 요청을 계속 대기
- 클라이언트 (Client)
 - create_client(타입, 이름) 생성 후 wait_for_service 로 준비 확인
 - Request 를 채워 call_async() 로 비동기 호출
 - future 로 응답(결과) 수신
- Python(rclpy)과 C++(rclcpp)의 구현 절차는 동일

빌드 & 실행

- 노드 실행

```
ros2 run cpp_srvcli server  
ros2 run cpp_srvcli client 2 3
```

- 실행 결과 : a=2, b=3 요청 → 서버가 sum=5 응답

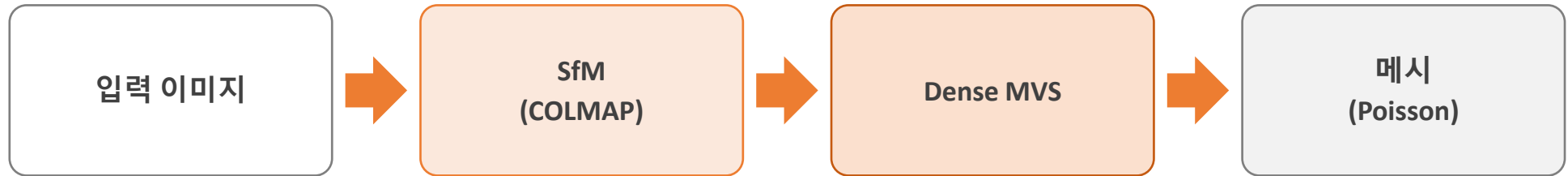
- CLI 로 빠른 테스트 (노드 작성 없이 호출)

```
ros2 service list  
ros2 service call /add_two_ints ...
```

- 인터페이스 확인 : ros2 interface show

전통적 SfM + MVS — 개요

- 여러 장의 2D 사진에서 카메라 포즈와 3D 형상을 복원 (이미지 기반 3D 재구성)



- SfM : 카메라 포즈 + 희소 점군 (sparse)
- Dense MVS : 깊이맵 융합으로 조밀 점군 (dense)
- Poisson : 점군의 법선으로 연속 표면(메시) 복원

SfM → MVS → 메시 : 데이터 흐름

- 각 단계의 산출물이 다음 단계의 입력이 되어 이어짐
- 이미지 → database.db
 - feature_extractor · matcher (특징 추출 · 매칭)
- database.db → sparse/ (포즈 + 희소 점군)
 - mapper + Bundle Adjustment (SfM 희소 복원)
- sparse/ → 보정 이미지 → depth / normal maps
 - image_undistorter → patch_match_stereo
- depth maps → fused.ply (조밀 점군) → meshed.ply (3D 메시)
 - stereo_fusion → poisson_mesher

SfM (COLMAP) — 희소 복원

- 특징 검출 · 매칭
 - SIFT 등으로 특징점 추출, 이미지 간 대응점 + 기하 검증
- 증분식 등록
 - 이미지를 하나씩 등록하며 카메라 내·외부 파라미터(포즈) 추정
- 삼각측량
 - 대응점을 3D 공간으로 역투영하여 희소 점(3D point) 생성
- Bundle Adjustment
 - 재투영 오차를 최소화하도록 포즈와 3D 점을 전역 최적화
- 출력 : 카메라 포즈 + 희소 점군 → MVS 입력

Dense MVS — 조밀 복원

- 왜곡 보정 (Undistortion)
 - 렌즈 왜곡 제거로 스테레오 매칭을 단순화
- PatchMatch Stereo
 - 이미지별 깊이맵 · 법선맵 계산
 - 신뢰 높은 시드에서 깊이 가설 전파, 픽셀별 뷰 선택으로 가림 회피
- Stereo Fusion
 - 여러 깊이맵을 일관성 검사 후 3D로 융합 → 조밀 점군 (fused.ply)
- 연산량이 커 GPU(CUDA) 가속이 강력히 권장됨

Poisson 메시 재구성

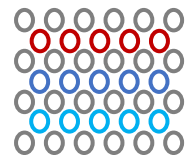
- 조밀 점군의 위치와 법선(normal)으로 연속적인 3D 표면을 복원
- Screened Poisson 표면 재구성
 - 법선장을 가장 잘 만족하는 음함수를 풀어 메시 생성
 - 빈 영역도 메워 watertight 표면에 적합
- 주요 파라미터
 - depth : 옥트리 최대 깊이 (클수록 디테일 ↑ · 메모리 ↑)
 - point_weight : 점 보간 가중치
- 대안 : Delaunay meshing (CGAL) → QEM 단순화 · 텍스처 매핑
- 결과 : 사진 → 포즈 → 점군 → 최종 3D 메시 모델

정리

- 커스텀 인터페이스
 - msg / srv / action, IDL → 언어별 코드 자동 생성
- Service 통신
 - 동기 1:1 요청/응답, create_service · create_client
- 전통적 SfM + MVS
 - SfM(희소) → Dense MVS(조밀) → Poisson(메시)

Q & A

Thank you for your attention



Architecture and Compiler
for Embedded Systems Lab.

Graduate School of Electronic and Electrical Engineering, KNU
ACE Lab. (hw051656@gmail.com)